

TP 3 — Simulateur de Centre de Distribution LogiTech Express

Alexis PICOT

TP 3 — LogiTech Express : Simulateur de Centre de Distribution

Contexte

Vous êtes développeur dans une startup de logistique qui tente de concurrencer les géants du secteur. Votre CTO vous confie le développement d'un simulateur pour optimiser les opérations d'un centre de distribution.

Il vous présente le projet :

“On a levé des fonds, on va ouvrir notre premier entrepôt. Mais avant de se lancer, on veut simuler le fonctionnement pour identifier les goulots d'étranglement. L'idée c'est simple : des colis arrivent, on les trie, on les charge dans des véhicules, et on les livre. Je veux un simulateur qui tourne en console et qui me donne des stats.”

Votre CTO est un ancien consultant. Il adore les buzzwords et change de vision stratégique toutes les deux semaines. Accrochez-vous.

Étape 1 — Le MVP (Minimum Viable Product)

Votre CTO vous envoie un premier brief sur Slack :

“On commence petit. Un colis a une destination et un poids. Il arrive à l'entrepôt, on le met dans un camion, le camion part livrer quand il est plein. C'est tout. Just ship it!”

Ce que vous devez réaliser

Créez un programme Java qui simule :

1. **Arrivée de colis**
 - Chaque colis a un identifiant unique
 - Chaque colis a une destination (ville)
 - Chaque colis a un poids (en kg)
2. **Gestion de l'entrepôt**
 - Les colis arrivent et sont stockés temporairement
 - Afficher les colis en attente
3. **Véhicules de livraison**
 - Un camion a une capacité maximale (en kg)
 - Créer au moins 3 camions avec des capacités différentes
4. **Chargement**
 - Charger des colis dans un camion
 - Vérifier que le poids total ne dépasse pas la capacité
 - Un colis chargé n'est plus dans l'entrepôt
5. **Livraison**
 - Quand un camion est suffisamment chargé (> 80% de sa capacité), il part en livraison
 - Après un certain temps (simulé), il revient vide

6. Statistiques

- Nombre de colis traités
- Nombre de livraisons effectuées
- Poids total livré

Simulation du temps

Pour simuler le temps qui passe, vous pouvez : - Soit utiliser un système de “tours” où chaque tour représente une unité de temps - Soit utiliser `Thread.sleep()` pour des délais réels (mais plus lent à tester)

Questions à vous poser AVANT de coder

- Comment représenter un colis ?
- Comment représenter un camion ?
- Comment gérer la file d’attente des colis ?
- Comment suivre l’état d’un camion (en entrepôt, en livraison) ?
- Où stocker les statistiques ?

Prenez 15 minutes pour concevoir votre modèle avant de coder.

Étape 2 — “On diversifie les véhicules”

Réunion stratégique. Votre CTO a eu une illumination :

“Les camions c’est bien, mais c’est pas scalable ! On va avoir des vélos-cargos pour les livraisons urbaines rapides, des drones pour les colis légers et urgents, et des trains de marchandises pour les gros volumes inter-régionaux. Chaque type de véhicule a ses contraintes !”

Nouveaux types de véhicules

1. **Camion** (ce que vous avez déjà)
 - Capacité : 1000 kg
 - Destinations : toutes
 - Temps de livraison : 4 heures
 - Coût au km : 0.50€
2. **Vélo-cargo**
 - Capacité : 50 kg
 - Destinations : même ville uniquement
 - Temps de livraison : 1 heure
 - Coût au km : 0.10€
 - Contrainte : un seul colis à la fois (pas de groupage)
3. **Drone**
 - Capacité : 5 kg
 - Destinations : rayon de 10 km seulement
 - Temps de livraison : 30 minutes
 - Coût au km : 0.30€
 - Contrainte : météo — ne vole pas s’il pleut (à simuler)
4. **Train**
 - Capacité : 50 000 kg
 - Destinations : villes avec gare uniquement
 - Temps de livraison : 8 heures
 - Coût au km : 0.05€
 - Contrainte : horaires fixes (départ à 6h, 14h, 22h)

Affectation des colis

- Le système doit choisir automatiquement le véhicule le plus adapté selon :
 - Le poids du colis
 - La destination
 - L’urgence (nouveau : certains colis sont “express”)
 - La disponibilité des véhicules

Ce que vous devez adapter

- Gérer les différents types de véhicules avec leurs contraintes spécifiques
- Implémenter la logique d’affectation
- Gérer les contraintes particulières (météo pour les drones, horaires pour les trains)
- Calculer les coûts de livraison

Questions à vous poser

- Comment représenter les différents types de véhicules ?
- Comment gérer les contraintes spécifiques à chaque type ?
- Comment implémenter l’algorithme de choix du véhicule ?
- Comment éviter les `if/else` géants sur le type de véhicule ?

Si vous avez un `switch` avec “CAMION”, “VELO”, “DRONE”, “TRAIN”, vous êtes dans le piège.

Étape 3 — “On scale ! Et on gère les problèmes !”

Nouveau pivot stratégique :

“On a décroché un contrat avec une grosse marketplace ! Ça veut dire : beaucoup plus de colis, plusieurs entrepôts dans différentes villes, et surtout des INCIDENTS à gérer. Des colis perdus, des véhicules en panne, des retards... Le client veut du tracking en temps réel et des notifications. Et bien sûr, il faut optimiser les coûts !”

Nouvelles fonctionnalités

Multi-entrepôts

- 3 entrepôts dans 3 villes différentes
- Les colis arrivent à l’entrepôt le plus proche de l’expéditeur
- Certains colis doivent transiter par un autre entrepôt si le véhicule direct n’est pas optimal
- Transferts inter-entrepôts par train

Gestion des incidents

1. **Colis endommagé**
 - Probabilité : 2% à chaque manipulation
 - Impact : remboursement au client
 - Action : créer un rapport d’incident
2. **Véhicule en panne**
 - Probabilité : 5% par livraison
 - Impact : véhicule indisponible pendant X heures
 - Action : réaffecter les colis à d’autres véhicules
3. **Retard de livraison**
 - Probabilité : 10% (trafic, intempéries...)
 - Impact : notification au client

- Action : mise à jour du tracking
4. **Colis perdu**
- Probabilité : 0.5%
 - Impact : remboursement + enquête
 - Action : créer un rapport d'incident critique

Système de tracking

- Chaque colis a un historique d'événements :
 - Réception à l'entrepôt
 - Chargement dans véhicule
 - En cours de livraison
 - Livré / Incident
- Le client peut consulter l'état de son colis à tout moment

Notifications

- Notifier le client (simulé par affichage console) :
 - Quand le colis est expédié
 - Quand le colis est en cours de livraison
 - Quand le colis est livré
 - En cas de retard ou problème

Optimisation des coûts

- Choisir le véhicule qui minimise le coût total
- Grouper les colis vers la même destination
- Équilibrer charge et coût

Ce que vous devez implémenter

- Architecture multi-entrepôts avec transferts
- Gestion des incidents et récupération
- Système de tracking avec historique
- Système de notifications
- Calcul et optimisation des coûts

Questions à vous poser

- Comment gérer les transitions d'état d'un colis (reçu → chargé → en livraison → livré/incident) ?
- Comment éviter que la gestion des incidents soit dispersée partout ?
- Comment rendre le système de notification extensible (email, SMS, app mobile...) ?
- Comment centraliser le calcul des coûts ?

Votre code commence peut-être à ressembler à une pelote de laine. C'est le moment de refactoriser.

Attendus fonctionnels

À la fin du TP, votre simulateur doit permettre de :

Gestion des colis

- ☐ Créer des colis avec destination, poids, urgence
- ☐ Recevoir des colis dans un entrepôt

- ☐ Suivre l'état d'un colis (tracking)
- ☐ Consulter l'historique d'un colis

Gestion des véhicules

- ☐ Gérer différents types de véhicules (camion, vélo, drone, train)
- ☐ Respecter les contraintes de chaque type
- ☐ Suivre l'état des véhicules (disponible, en livraison, en panne)
- ☐ Gérer les pannes et réparations

Gestion des entrepôts

- ☐ Multi-entrepôts dans différentes villes
- ☐ Transferts inter-entrepôts
- ☐ Affichage de l'état de chaque entrepôt

Affectation et livraison

- ☐ Affecter automatiquement le meilleur véhicule
- ☐ Charger les colis en respectant les capacités
- ☐ Simuler les livraisons avec temps de trajet
- ☐ Gérer les incidents aléatoires

Statistiques et reporting

- ☐ Nombre de colis traités par période
- ☐ Taux d'incidents
- ☐ Coûts de livraison
- ☐ Performance par type de véhicule
- ☐ Performance par entrepôt

Interface console

- ☐ Menu principal clair
- ☐ Visualisation de l'état du système
- ☐ Simulation pas à pas ou automatique

Pièges classiques dans lesquels vous allez probablement tomber

Piège n°1 : La classe Véhicule fourre-tout

Comme pour le TP 1, vous pourriez créer une seule classe `Vehicule` avec un attribut `type` :

```
Vehicule:
- type: String ("CAMION", "VELO", "DRONE", "TRAIN")
- capacite: int
- peutVoler: boolean (true seulement pour drone)
- aDesHoraires: boolean (true seulement pour train)
- horaireDepart: List<LocalTime> (null si pas train)
- rayonMax: int (0 si pas drone)
- ...
```

Pourquoi c'est un piège ? - Beaucoup d'attributs inutiles selon le type - La logique de vérification des contraintes devient un cauchemar de if/else - Impossible d'ajouter un nouveau type de véhicule sans tout modifier

Point pédagogique pour l'enseignant : C'est le moment de montrer la puissance de l'héritage combiné aux **méthodes abstraites**. Une classe abstraite **Vehicule** définit le contrat (`peutTransporter(Colis c)`, `getCoutLivraison()`, etc.), et chaque type de véhicule implémente sa propre logique.

Piège n°2 : L'état du colis géré par des booléens

```
class Colis {
    boolean estRecu = false;
    boolean estCharge = false;
    boolean estEnLivraison = false;
    boolean estLivre = false;
    boolean estEndommage = false;
    boolean estPerdu = false;
}
```

Et ensuite, des validations partout :

```
if (colis.estRecu && !colis.estCharge && !colis.estEnLivraison && !colis.estLivre) {
    // On peut le charger
}
```

Pourquoi c'est un piège ? - Combinatoire explosive de conditions - États incohérents possibles (livré ET perdu ?) - Transitions d'état non contrôlées - Difficile à maintenir et à déboguer

Point pédagogique pour l'enseignant : C'est le moment d'introduire le pattern **State** ou l'utilisation d'**énumérations** pour modéliser une machine à états. Le colis a un état unique à tout moment, et les transitions sont contrôlées.

Piège n°3 : La gestion des incidents dispersée

Vous pourriez avoir du code de gestion d'incidents partout :

```
// Dans Vehicule.java
public void livrer() {
    if (Math.random() < 0.05) {
        this.enPanne = true;
        System.out.println("Véhicule en panne !");
        // Logique de réaffectation ici ?
    }
    // ...
}
```

```
// Dans Colis.java
public void charger() {
    if (Math.random() < 0.02) {
        this.endommage = true;
        System.out.println("Colis endommagé !");
        // Logique de remboursement ici ?
    }
}
```

Pourquoi c'est un piège ? - La logique de gestion des incidents est éparpillée - Difficile de faire des statistiques sur les incidents - Difficile de changer la façon dont on gère un type d'incident - Couplage fort entre les classes métier et la gestion d'erreurs

Point pédagogique pour l'enseignant : C'est le moment d'introduire les **exceptions personnalisées** pour les incidents, et/ou le pattern **Observer** pour notifier un gestionnaire

d'incidents centralisé.

Piège n°4 : Le tracking comme liste de strings

```
class Colis {  
    List<String> historique = new ArrayList<>();  
  
    public void ajouterEvenement(String event) {  
        historique.add(LocalDate.now() + " - " + event);  
    }  
}
```

```
colis.ajouterEvenement("Reçu à l'entrepôt Lyon");  
colis.ajouterEvenement("Chargé dans camion C-001");
```

Pourquoi c'est un piège ? - Pas de structure, impossible de faire des requêtes - Pas de typage, facile de faire des fautes - Impossible de filtrer par type d'événement - Difficile d'ajouter des métadonnées (qui a fait l'action, durée...)

Point pédagogique pour l'enseignant : C'est le moment de créer une vraie classe **EvenementTracking** avec un type (enum), une date, des détails structurés. On peut même créer des sous-classes pour différents types d'événements.

Piège n°5 : L'algorithme de choix du véhicule monolithique

```
public Vehicule choisirVehicule(Colis colis) {  
    if (colis.getPoids() <= 5 && colis.getDestination().isLocal() && !ilPleut()) {  
        // Chercher un drone disponible  
        for (Vehicule v : vehicules) {  
            if (v.getType().equals("DRONE") && v.isDisponible()) {  
                return v;  
            }  
        }  
    }  
    if (colis.getPoids() <= 50 && colis.getDestination().isMemeVille()) {  
        // Chercher un vélo  
        // ...  
    }  
    // etc. sur 200 lignes  
}
```

Pourquoi c'est un piège ? - Méthode gigantesque et illisible - Ajouter un nouveau type de véhicule = modifier cette méthode - Difficile à tester unitairement - Logique de scoring/priorité mélangée avec la recherche

Point pédagogique pour l'enseignant : C'est le moment d'introduire le pattern **Strategy** ou **Chain of Responsibility**. Chaque type de véhicule peut dire s'il peut transporter le colis et à quel coût. L'algorithme de choix devient simple : demander à chaque véhicule et garder le meilleur.

Étapes facultatives (différenciation pédagogique)

Niveau 1 — Pour ceux qui terminent l'étape 3

Extension 1.1 : Créneaux de livraison

“Les clients veulent choisir leur créneau de livraison ! Matin (8h-12h), après-midi (14h-18h), ou soir (18h-22h). Il faut en tenir compte dans l’affectation des véhicules.”

Ce que vous devez implémenter : - Choix de créneau par colis - Les véhicules ont des contraintes horaires - Vérification que le véhicule peut livrer dans le créneau demandé - Gestion des colis “livrables quand vous voulez”

Extension 1.2 : Colis fragiles et dangereux

“On a des types de colis spéciaux : fragiles (manipulation délicate, surcoût) et dangereux (matières chimiques, pas de drone, pas de vélo, réglementation spéciale).”

Ce que vous devez implémenter : - Types de colis avec contraintes spéciales - Surcoût pour manipulation spéciale - Restrictions de véhicules selon le type de colis - Documentation automatique pour les colis dangereux

Niveau 2 — Pour ceux qui vont vite

Extension 2.1 : Simulation météo et trafic

“La météo et le trafic impactent les livraisons. Je veux une simulation réaliste avec des conditions qui changent au cours de la journée.”

Ce que vous devez implémenter : - Simulation météo (beau, nuageux, pluie, neige, tempête) - Impact météo sur les véhicules (drones cloués au sol, camions ralentis...) - Simulation trafic aux heures de pointe - Ajustement automatique des temps de livraison - Historique météo/trafic pour les stats

Question piège : Comment faire en sorte que la météo affecte les véhicules sans modifier chaque classe de véhicule ?

Point pédagogique pour l’enseignant : C’est le moment d’introduire le principe d’**injection de dépendances**. Un service météo est injecté dans le système et les véhicules le consultent pour savoir s’ils peuvent opérer.

Extension 2.2 : Tableau de bord temps réel

“Je veux un dashboard qui s’affiche en continu avec l’état du système, les véhicules en mouvement, les colis en transit, les alertes...”

Ce que vous devez implémenter : - Affichage rafraîchi périodiquement - Visualisation ASCII art de la carte avec entrepôts et véhicules - Liste des alertes actives - KPIs en temps réel (colis/heure, taux de livraison, coût moyen...) - Historique des derniers événements

Niveau 3 — Pour les très rapides

Extension 3.1 : Optimisation par lots

“On veut regrouper les colis vers la même zone pour optimiser les tournées. Un camion fait une tournée de 5-10 livraisons au lieu d’un aller-retour par colis.”

Ce que vous devez implémenter : - Algorithme de regroupement par zone géographique - Calcul de la tournée optimale (problème du voyageur de commerce simplifié) - Temps de livraison par tournée et non par colis - Optimisation du remplissage des véhicules

Question piège : Comment résoudre le problème du voyageur de commerce ? (Indice : une solution approximative suffit !)

Point pédagogique pour l’enseignant : C’est le moment d’introduire les **algorithmes gloutons** et les **heuristiques**. On peut aussi parler de récursivité pour une solution exacte sur de petits problèmes.

Extension 3.2 : Gestion des retours

“Les clients peuvent retourner des colis. Il faut organiser la collecte, le retour à l’entrepôt, et le remboursement.”

Ce que vous devez implémenter : - Processus de demande de retour - Collecte chez le client (véhicule qui passe récupérer) - Inspection à l’entrepôt - Remboursement (total, partiel, refusé) - Réintégration dans le stock ou destruction

Extension 3.3 : Mode simulation accélérée

“Je veux pouvoir simuler une semaine d’opérations en quelques secondes pour tester différentes configurations.”

Ce que vous devez implémenter : - Accélération du temps de simulation (x10, x100, x1000) - Génération automatique de colis selon des patterns réalistes - Comparaison de configurations (nombre de véhicules, capacités...) - Export des stats en CSV pour analyse

Limites pédagogiques à annoncer aux étudiants

Ce qui est INTERDIT

1. **Pas de switch sur les types de véhicules** Le choix du véhicule doit être polymorphique. Chaque véhicule sait s’il peut prendre un colis.
2. **Pas d’états incohérents** Un colis ne peut pas être “en livraison” et “livré” en même temps. Utilisez une vraie machine à états.
3. **Pas de code de gestion d’erreur dispersé** Les incidents doivent être gérés de manière centralisée ou via des exceptions.
4. **Pas de “managers” qui font tout** Évitez la classe `LogistiqueManager` de 2000 lignes. Répartissez les responsabilités.
5. **Pas de duplication des règles métier** Si la capacité d’un camion est vérifiée à 5 endroits, c’est un problème. Centralisez.

Ce qui est OBLIGATOIRE

1. **Utiliser l’encapsulation** Les attributs sont privés. On ne fait pas `vehicule.colis.add(c)`.
 2. **Utiliser le polymorphisme** On doit pouvoir manipuler tous les véhicules de la même manière.
 3. **Gérer les erreurs proprement** Exceptions pour les cas exceptionnels, retours typés pour les cas normaux.
 4. **Documenter les choix de conception** Pourquoi avez-vous créé cette classe ? Cette interface ?
 5. **Tester les scénarios métier**
 - Colis normal : réception → affectation → livraison
 - Colis trop lourd pour un vélo
 - Drone avec pluie
 - Véhicule en panne pendant une livraison
-

Suggestions d'improvisation pour l'enseignant

Pendant l'étape 1

- “Et si un camion tombait en panne pendant la livraison ?”
- “Comment sauriez-vous où est un colis à tout moment ?”
- “Si j’ajoute des motos demain, c’est facile ?”

Pendant l'étape 2

- “Montrez-moi comment le drone vérifie qu’il peut voler (météo)”
- “Comment le train sait-il qu’il doit attendre le prochain horaire ?”
- “Comment ajouteriez-vous un bateau pour les livraisons portuaires ?”

Pendant l'étape 3

- “Un colis est signalé comme livré mais le client dit ne pas l’avoir reçu. Comment enquêtez-vous ?”
- “On veut envoyer un SMS au client en plus de l’affichage console. Combien de modifications ?”
- “Comment géreriez-vous un black-out dans un entrepôt ?”

Pour challenger les meilleurs

- “Comment feriez-vous pour prédire les pics de charge ?”
- “Comment optimiseriez-vous la répartition des colis entre entrepôts ?”
- “Comment géreriez-vous le cas où TOUS les véhicules sont occupés ?”

Questions de réflexion

- “Votre simulateur est-il déterministe ou aléatoire ? Pourquoi ce choix ?”
 - “Comment testeriez-vous que l’affectation des véhicules est correcte ?”
 - “Si je veux changer la formule de calcul des coûts, combien de fichiers dois-je modifier ?”
 - “Votre code pourrait-il tourner 24h sans intervention humaine ?”
-

Critères d'évaluation (indicatifs)

Fonctionnel (40%)

- La simulation fonctionne de bout en bout
- Les contraintes des véhicules sont respectées
- Les incidents sont gérés
- Les statistiques sont calculées correctement

Conception (35%)

- Modélisation claire des entités métier
- Utilisation appropriée de l’héritage et des interfaces
- Séparation des responsabilités
- Gestion propre des états

Qualité du code (15%)

- Code lisible et bien structuré
- Nommage explicite
- Pas de duplication
- Gestion des erreurs appropriée

Résilience et extensibilité (10%)

- Facilité à ajouter un nouveau type de véhicule
 - Facilité à ajouter un nouveau type d'incident
 - Le système reste stable en cas de problème
-

Pour bien démarrer

1. **Listez les entités métier** — Colis, Véhicule, Entrepôt, Livraison, Incident...
2. **Identifiez les états** — Un colis peut être dans quel état ? Un véhicule ?
3. **Dessinez les relations** — Qu'est-ce qui contient quoi ? Qui connaît qui ?
4. **Commencez par le flux nominal** — Un colis arrive, est chargé, est livré. Point.
5. **Ajoutez la complexité progressivement** — Types de véhicules, puis incidents, puis multi-entrepôts...

La logistique est un domaine complexe. N'essayez pas de tout faire d'un coup. Construisez brique par brique, testez à chaque étape, et refactorisez quand le code devient difficile à lire.

Bon courage, et que vos colis arrivent à bon port !

Annexe : Données de test suggérées

Villes et distances

Origine	Destination	Distance (km)	Gare ?
Lyon	Lyon	0	Oui
Lyon	Paris	470	Oui
Lyon	Marseille	315	Oui
Lyon	Villeurbanne	5	Non
Lyon	Grenoble	105	Oui
Paris	Marseille	775	Oui
Paris	Lille	225	Oui

Véhicules de départ

Type	ID	Capacité	Entrepôt
Camion	C-001	1000 kg	Lyon
Camion	C-002	1500 kg	Lyon
Camion	C-003	800 kg	Paris
Vélo	V-001	50 kg	Lyon
Vélo	V-002	50 kg	Paris
Drone	D-001	5 kg	Lyon
Drone	D-002	5 kg	Paris
Train	T-001	50000 kg	Lyon

Exemples de colis

ID	Poids	Origine	Destination	Urgent ?
COL-001	2 kg	Lyon	Villeurbanne	Oui

ID	Poids	Origine	Destination	Urgent ?
COL-002	30 kg	Lyon	Paris	Non
COL-003	500 kg	Lyon	Marseille	Non
COL-004	4 kg	Paris	Paris	Oui
COL-005	8000 kg	Lyon	Paris	Non

Utilisez ces données pour tester votre simulateur, puis créez vos propres scénarios !